

Примеры интеграций

Multichannel Hub отправляет события (например, `message.received`) на ваши вебхуки. Это позволяет связать шлюз с любыми системами автоматизации и CRM. Ниже — готовые примеры.

Формат события

На указанный URL приходит `POST`-запрос:

```
{
  "event": "message.received",
  "data": {
    "id": "uuid",
    "channelId": "uuid",
    "direction": "INBOUND",
    "senderName": "Иван Иванов",
    "senderId": "123456789",
    "text": "Текст обращения",
    "payload": { "...": "исходные данные канала" },
    "createdAt": "2026-06-29T12:00:00.000Z"
  },
  "timestamp": "2026-06-29T12:00:00.000Z"
}
```

Заголовки:

- `Content-Type: application/json`
- `X-Webhook-Event: message.received`
- `X-Webhook-Signature: <HMAC-SHA256>` — если для вебхука задан `secret`.

Проверка подписи (Node.js)

```
const crypto = require('crypto');

function verify(rawBody, signature, secret) {
  const expected = crypto.createHmac('sha256', secret).update(rawBody).digest('hex');
  return crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(signature));
}
```

n8n

1. Добавьте узел **Webhook** (метод `POST`), скопируйте его Production URL.
2. В админ-панели: **Вебхуки** → **Добавить вебхук**, вставьте URL, при желании задайте секрет.
3. Постройте сценарий: например, узел **Webhook** → **Set** → **CRM/Email/Telegram**.

Пример доступа к полям в n8n:

```
{{ $json.body.data.text }}
{{ $json.body.data.senderName }}
```

Make (Integromat)

1. Создайте сценарий, добавьте триггер **Webhooks** → **Custom webhook**, получите адрес.
2. Вставьте адрес в **Вебхуки** админ-панели.
3. Отправьте тестовое сообщение в Telegram-бота, чтобы Make «определил структуру данных».
4. Далее добавляйте модули (Google Sheets, CRM, Email и т.д.), используя поля из `data`.

Bitrix24

Вариант А — через открытую линию / входящий вебхук CRM:

1. В Bitrix24: **Разработчикам** → **Другое** → **Входящий вебхук**, выдайте право `crm`.
2. Получите URL вида `https://<portal>.bitrix24.ru/rest/<user>/<token>/`.
3. Чаще всего удобнее принять событие промежуточным сервисом (n8n/Make) и вызвать метод `crm.lead.add`. Пример тела запроса к Bitrix24:

```
{
  "fields": {
    "TITLE": "Обращение из Multichannel Hub",
    "NAME": "Иван Иванов",
    "COMMENTS": "Текст обращения",
    "SOURCE_ID": "WEB"
  }
}
```

Вариант Б — прямой вызов: укажите в качестве URL вебхука эндпоинт вашего обработчика, который преобразует событие в вызов `crm.lead.add`.

Zapier

1. Создайте Zap с триггером **Webhooks by Zapier** → **Catch Hook**.
2. Скопируйте URL и добавьте его в **Вебхуки** админ-панели.
3. Настройте действие (Action) в нужный сервис, используя поля `data.text`, `data.senderName` и др.

Приём заявок с веб-формы сайта

Создайте канал типа **Web Form** и отправляйте заявки на публичный эндпоинт:

```

<form id="lead-form">
  <input name="name" placeholder="Имя" />
  <input name="email" placeholder="Email" />
  <input name="phone" placeholder="Телефон" />
  <textarea name="message" placeholder="Сообщение"></textarea>
  <button type="submit">Отправить</button>
</form>

<script>
  const CHANNEL_ID = 'ВАШ_CHANNEL_ID';
  document.getElementById('lead-form').addEventListener('submit', async (e) => {
    e.preventDefault();
    const form = new FormData(e.target);
    await fetch(`https://your-domain.com/api/webforms/${CHANNEL_ID}/submit`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(Object.fromEntries(form)),
    });
    alert('Заявка отправлена!');
  });
</script>

```

Заявка попадёт в историю сообщений и будет переслана во все активные вебхуки.

Интеграция WhatsApp через API

WhatsApp работает через библиотеку **whatsapp-web.js** — неофициальный бесплатный API. Авторизация происходит через QR-код (как при подключении связанного устройства).

1. Создание и авторизация канала

```

# Создать канал WhatsApp
POST /api/channels
{
  "name": "WhatsApp Support",
  "type": "WHATSAPP",
  "config": {}
}
# Ответ: { "id": "channel-uuid-123", ... }

# Инициализировать клиент
POST /api/whatsapp/channels/channel-uuid-123/initialize

# Получить QR-код для сканирования
GET /api/whatsapp/channels/channel-uuid-123/qr
# Ответ: { "qr": "data:image/png;base64,...", "status": "WAITING_QR" }

```

Отсканируйте QR-код в приложении WhatsApp:

Настройки → Связанные устройства → Привязать устройство.

2. Проверка статуса подключения

```

GET /api/whatsapp/channels/channel-uuid-123/status
# Ответ: { "status": "CONNECTED", "info": { ... } }

```

3. Отправка сообщений

```
POST /api/whatsapp/channels/channel-uuid-123/send
{
  "phoneNumber": "79001234567@с.us",
  "text": "Здравствуйте! Спасибо за обращение."
}
```

Формат номера: [countryCode][number]@с.us (например, 79001234567@с.us для России).

4. Приём входящих сообщений

Входящие сообщения из WhatsApp автоматически:

- Сохраняются в базу данных (таблица `messages`)
- Доступны в API `/api/messages`
- Отправляются на все активные вебхуки с событием `message.received`

Пример вебхука:

```
{
  "event": "message.received",
  "timestamp": "2026-06-29T12:00:00Z",
  "data": {
    "id": "msg-uuid",
    "channelId": "channel-uuid-123",
    "channelType": "WHATSAPP",
    "direction": "INBOUND",
    "senderId": "79001234567@с.us",
    "senderName": "Иван Иванов",
    "text": "Привет! Как дела?",
    "payload": { ... }
  }
}
```

5. Автоматизация через n8n / Make

n8n пример:

1. Webhook Trigger → получение событий `message.received` от Multichannel Hub
2. Switch Node → фильтрация по `data.channelType === 'WHATSAPP'`
3. HTTP Request → отправка ответа через `/api/whatsapp/channels/{id}/send`

Make пример:

1. Custom Webhook → подписка на события Multichannel Hub
2. Router → фильтр по типу канала
3. HTTP Module → отправка сообщения в WhatsApp через API

⚠ Важно: Сессии WhatsApp сохраняются в папке `whatsapp-sessions/` . При использовании Docker рекомендуется монтировать эту папку как volume, чтобы не терять авторизацию при перезапуске контейнера.

Интеграция Telegram (QR / пользовательский аккаунт) через API

Режим работает через **MTProto** (библиотека GramJS) — это полноценный клиент Telegram с авторизацией

по QR-коду, без регистрации бота. Подходит, когда нужно вести переписку от лица реального аккаунта.

1. Получение API ID / API Hash

Каждый пользователь получает свои ключи на my.telegram.org (<https://my.telegram.org>) →

API development tools. Эти `api_id` и `api_hash` указываются в настройках канала (каждый пользователь настраивает под свой аккаунт).

2. Создание канала и авторизация

```
# Создать канал (тип TELEGRAM, режим userbot)
POST /api/channels
{
  "name": "Telegram QR Support",
  "type": "TELEGRAM",
  "config": { "mode": "userbot", "apiId": 1234567, "apiHash": "0123456789abcde-
f0123456789abcdef" }
}

# Инициализировать клиент
POST /api/telegram-user/channels/{channelId}/initialize

# Получить QR-код (data URL PNG)
GET /api/telegram-user/channels/{channelId}/qr
```

Отсканируйте QR-код в Telegram: **Настройки → Устройства → Подключить устройство**. Если включён облачный пароль (2FA), передайте его:

```
POST /api/telegram-user/channels/{channelId}/password
{ "password": "my-2fa-password" }
```

3. Отправка сообщений

```
POST /api/telegram-user/channels/{channelId}/send
{
  "peer": "@username",
  "text": "Здравствуйте! Чем можем помочь?"
}
```

4. Приём входящих сообщений

Входящие сообщения автоматически сохраняются в БД и отправляются на вебхуки с событием `message.received`. Структура полностью совпадает с другими каналами (`channelType`: "TELEGRAM").

5. Автоматизация через n8n / Make

1. Webhook Trigger → получение событий `message.received`
2. Фильтр по `data.channelType === 'TELEGRAM'`

3. HTTP Request → ответ через `/api/telegram-user/channels/{id}/send`

⚠ **Важно:** Строка сессии сохраняется в БД (`config.session` канала) — авторизация переживает перезапуск. Автоматизация пользовательского аккаунта формально нарушает ToS Telegram: избегайте массовых рассылок и спама, иначе аккаунт может быть заблокирован.

Интеграция Discord через API

1. Создание Discord-бота

1. Откройте [Discord Developer Portal](https://discord.com/developers/applications) (<https://discord.com/developers/applications>) и создайте приложение (**New Application**).
2. В разделе **Bot** нажмите **Add Bot**, затем **Reset Token** — скопируйте **Bot Token**.
3. **Важно:** В **Bot** → **Privileged Gateway Intents** включите **Message Content Intent**.
4. В **OAuth2** → **URL Generator** выберите scope **bot**, permissions **Send Messages, Read Message History, View Channels** — скопируйте URL и добавьте бота на сервер.

2. Создание канала и подключение бота

```
POST /api/channels
Authorization: Bearer <token>
{
  "name": "Discord Bot",
  "type": "DISCORD",
  "config": {
    "botToken": "MTk4NjIyNDgzNDcxOTI1MjQ4.G..."
  }
}
```

Инициализация бота:

```
POST /api/discord/channels/{channelId}/initialize
```

Проверка статуса:

```
GET /api/discord/channels/{channelId}/status
# Ответ: { "status": "CONNECTED" }
```

3. Отправка сообщений

```
POST /api/discord/channels/{channelId}/send
{
  "discordChannelId": "123456789012345678",
  "text": "Привет из омниканального шлюза!"
}
```

💡 **Discord Channel ID** можно скопировать в Discord: ПКМ на канале → Copy ID (требуется Developer Mode в настройках).

4. Приём входящих сообщений

Бот автоматически получает сообщения из всех каналов Discord-серверов, куда добавлен.

Входящие сообщения:

- Сохраняются в БД с `direction: INBOUND`, `senderName` (Discord tag), `senderId` (Discord user ID).
- Триггерят вебхуки с событием `message.received` и полезной нагрузкой:

```
json
{
  "event": "message.received",
  "data": {
    "message": { "id": "...", "text": "...", "senderName": "User#1234", ... },
    "channel": { "id": "...", "name": "Discord Bot", "type": "DISCORD" },
    "platform": "discord"
  },
  "timestamp": "2024-06-29T12:00:00.000Z"
}
```

5. Автоматизация через n8n / Make

n8n: Webhook → HTTP Request → Discord send

```
// В n8n Webhook ноде получаете данные из шлюза
const message = $json.data.message;
const senderName = message.senderName;

// Отправка ответа обратно в Discord
$http.post('/api/discord/channels/{channelId}/send', {
  discordChannelId: "123456789012345678",
  text: `Привет, ${senderName}! Ваше сообщение получено.`
});
```

Интеграция Slack через API

1. Создание Slack-приложения

1. Откройте [Slack API](https://api.slack.com/apps) (<https://api.slack.com/apps>) → **Create New App** → **From scratch**.
2. Задайте имя и workspace.
3. В **OAuth & Permissions** → **Bot Token Scopes** добавьте: `chat:write`, `channels:history`, `groups:history`, `im:history`.
4. Нажмите **Install to Workspace** — скопируйте **Bot User OAuth Token** (начинается с `xoxb-`).
5. (Опционально, для получения входящих):
 - В **Socket Mode** включите Socket Mode → создайте **App-Level Token** (scope: `connections:write`, начинается с `xapp-`).
 - В **Basic Information** скопируйте **Signing Secret**.

2. Создание канала и подключение бота

Минимальная конфигурация (только отправка):

```
POST /api/channels
Authorization: Bearer <token>
{
  "name": "Slack Bot",
  "type": "SLACK",
  "config": {
    "botToken": "xoxb-..."
  }
}
```

Полная конфигурация (отправка + приём через Socket Mode):

```
{
  "name": "Slack Bot",
  "type": "SLACK",
  "config": {
    "botToken": "xoxb-...",
    "signingSecret": "abc123...",
    "appToken": "xapp-..."
  }
}
```

Инициализация бота:

```
POST /api/slack/channels/{channelId}/initialize
```

Проверка статуса:

```
GET /api/slack/channels/{channelId}/status
# Ответ: { "status": "CONNECTED_SOCKET_MODE" } или "CONNECTED_API_ONLY"
```

3. Отправка сообщений

```
POST /api/slack/channels/{channelId}/send
{
  "slackChannelId": "C1234567890",
  "text": "Привет из омниканального шлюза!"
}
```

💡 **Slack Channel ID** можно найти в адресной строке браузера: <https://app.slack.com/client/T.../C1234567890>.

4. Приём входящих сообщений (Socket Mode)

Если указаны `appToken` и `signingSecret`, бот работает в **Socket Mode** и получает входящие сообщения. Входящие сообщения:

- Сохраняются в БД с `direction: INBOUND`, `senderId` (Slack user ID).
- Триггерят вебхуки с событием `message.received` и полезной нагрузкой:

```
json
{
  "event": "message.received",
  "data": {
    "message": { "id": "...", "text": "...", "senderId": "U1234567890", ... },
```



```

    "channel": { "id": "...", "name": "Slack Bot", "type": "SLACK" },
    "platform": "slack"
  },
  "timestamp": "2024-06-29T12:00:00.000Z"
}

```

5. Автоматизация через n8n / Make

n8n: Webhook → HTTP Request → Slack send

```

// В n8n Webhook ноде получаете данные из шлюза
const message = $json.data.message;
const senderId = message.senderId;

// Отправка ответа обратно в Slack
$http.post('/api/slack/channels/{channelId}/send', {
  slackChannelId: "C1234567890",
  text: `<@${senderId}> Ваше сообщение получено!`
});

```

Интеграция VK (ВКонтакте) через API

1. Получение Access Token ВКонтакте

1. Перейдите на vk.com/dev (<https://vk.com/dev>).
2. Создайте **Standalone-приложение**.
3. Получите **Access Token** личного аккаунта с правами:
 - **messages** (доступ к сообщениям)
 - **offline** (бессрочный токен)
4. Токен выглядит примерно так: `vk1.a.AbCdEf1234567890...`

⚠ Важно: Используйте токен **личного аккаунта** (User Token), а не группы (Group Token). User Token позволяет отправлять и получать личные сообщения от имени вашего аккаунта.

2. Создание канала и подключение

Создание VK-канала:

```

POST /api/channels
Authorization: Bearer <token>
{
  "name": "VK Personal",
  "type": "VK",
  "config": {
    "accessToken": "vk1.a.AbCdEf..."
  }
}

```

Инициализация клиента (запуск User Long Poll):

```

POST /api/vk/channels/{channelId}/initialize

```

Проверка статуса:

```
GET /api/vk/channels/{channelId}/status
# Ответ: { "status": "CONNECTED" }
```

3. Отправка сообщений

```
POST /api/vk/channels/{channelId}/send
{
  "userId": 123456789,
  "text": "Привет из омниканального шлюза!"
}
```

💡 **VK User ID** можно узнать через профиль ВК или методом `users.get` VK API.

4. Приём входящих сообщений (User Long Poll)

VK-клиент автоматически получает входящие личные сообщения через **User Long Poll**.

Входящие сообщения:

- Сохраняются в БД с `direction: INBOUND`, `senderId` (VK user ID), `senderName` (VK User {id}).
- Триггерят вебхуки с событием `message.received` и полезной нагрузкой:

```
json
{
  "event": "message.received",
  "data": {
    "message": {
      "id": "...",
      "text": "...",
      "senderId": "123456789",
      "senderName": "VK User 123456789",
      "payload": {
        "conversationMessageId": 123,
        "peerId": 123456789,
        "fromId": 123456789,
        "date": 1719662400,
        "attachments": []
      },
    },
    ...
  },
  "channel": { "id": "...", "name": "VK Personal", "type": "VK" },
  "platform": "vk",
  "timestamp": "2024-06-29T12:00:00.000Z"
}
```

5. Автоматизация через n8n / Make

n8n: Webhook → HTTP Request → VK send

```
// В n8n Webhook нодe получаете данные из шлюза
const message = $json.data.message;
const senderId = parseInt(message.senderId);

// Отправка ответа обратно пользователю ВК
$http.post('/api/vk/channels/{channelId}/send', {
  userId: senderId,
  text: `Ваше сообщение "${message.text}" получено!`
});
```

Make (Integromat): Custom Webhook → HTTP Module → VK send

1. Настройте **Custom Webhook** для получения события `message.received`.
2. Используйте **HTTP module** с методом `POST` на `/api/vk/channels/{channelId}/send`.
3. Отправьте `userId` и `text` в теле запроса.

Добавление новых каналов (для разработчиков)

Архитектура модульная. Чтобы добавить канал (например, Instagram или MAX):

1. Добавьте значение в enum `ChannelType` в `backend/prisma/schema.prisma` и создайте миграцию.
2. Создайте новый модуль в `backend/src/<channel>/` по образцу `telegram/` или `whatsapp/`.
3. Реализуйте приём входящих сообщений через `MessagesService.createInbound(...)` — это автоматически запустит доставку в вебхуки.
4. При необходимости добавьте тип канала в форму создания на фронтенде (`frontend/src/app/(panel)/channels/page.tsx`).

Подробнее см. [AGENTS.md](#) (./AGENTS.md) — руководство для разработчиков и AI-агентов.